

Optimization Uncertainty in Learning for Income Prediction and Wine Quality Classification

Syed Muhammad Husainie

Georgia Institute of Technology

CS 7641: Machine Learning (Spring 2026)

shusainie3

Link: <https://www.overleaf.com/read/kmzjr bwgdccf19d3df>

Abstract—For this study, optimization strategies were implemented to measure their impact on neural networks. Two benchmark classification datasets were used; Adult Income and Wine Quality. The Pytorch multilayer architecture implemented previously was used as baseline, and different training mechanisms were examined on it. First, randomized optimization methods including Randomized Hill Climbing, Simulated Annealing, and Genetic Algorithms were applied to the final network layers while freezing all other layers. The parameter budget was also restricted to analyze each algorithms ability to escape local minima and improve validation objectives. Second, optimizer ablation study was carried out to analyze Adam variants and the contributions of momentum, bias, adaptive scaling, and decoupled weight decay. Lastly, multiple regularization techniques were evaluated to highlight the trade offs between convergence speed, seed stability, and final test performance; leading to a practical training recipe for neural network classification tasks.

I. INTRODUCTION

This study investigates how different optimization strategies influence the training dynamics and generalization performance of neural networks under controlled experimental conditions. The main goal is to understand whether modifications to the optimization procedures can improve performance, stability, or convergence behavior - without altering the baseline architecture. Experiments are conducted on two benchmarks: the Adult Income dataset for binary classification and the Wine Quality dataset for multiclass classification.

The investigation focuses on three optimization strategies. First, randomized optimization (RO) techniques including Randomized Hill Climbing (RHC), Simulated Annealing (SA), and Genetic Algorithms (GA) are applied to the final layers of the neural network under strict parameter constraints; allowing a controlled comparison between zero-order search methods and gradient-based learning within a limited parameter space. Second, a series of optimizer ablations is performed on the Adult dataset to isolate the contributions of momentum, adaptive scaling, bias correction, and decoupled weight decay within modern stochastic optimization methods. Finally, regularization techniques are evaluated to determine how explicit regularization interacts with optimizer behavior and influences generalization performance. All experiments use identical train/validation/test splits, budgets, batch sizes, and hardware from the SL report to ensure consistency.

II. DATASETS AND EXPLORATORY DATA ANALYSIS

An exploratory data analysis was carried out for both datasets to help guide optimization hypotheses: Adult Income and Wine Quality. These datasets were previously used in the supervised learning report and therefore maintain the same preprocessing pipeline and train-validation-test splits to ensure comparability across experiments.

A. Adult Income Dataset

The Adult Income dataset involves predicting whether an individual's annual income exceeds \$50,000 based on demographic and employment-related attributes. The dataset contains 45,175 observations after removing duplicate rows, with a total of 15 features including both categorical and numerical attributes.

Several important characteristics are noted from the exploratory analysis. First, the dataset contains a mix of categorical and continuous variables, requiring encoding and normalization before training neural network models. Second, several numerical variables such as *capital-gain* and *capital-loss* have heavy right skew due to a large number of zero values and a small number of extreme observations. Third, the target variable has moderate class imbalance, with approximately 75% of instances belonging to the $\leq 50K$ class and 25% belonging to the $> 50K$ class.

These properties make the Adult dataset a useful benchmark for studying optimization stability under class imbalance and skewed feature distributions. In particular, adaptive optimizers and appropriate regularization techniques may help mitigate instability caused by these characteristics.

B. Wine Quality Dataset

The Wine Quality dataset focuses on predicting wine quality scores based on physicochemical measurements of wine samples. This task represents a multiclass classification problem with continuous input features and moderate label imbalance across quality categories. The dataset consists of 5,320 unique observations after removing duplicate rows and includes 14 numerical attributes describing characteristic properties of wine samples, with quality ratings scale ranging from 1 to 8.

EDA reveals several important properties of this dataset. Most features are continuous chemical measurements with

moderate right skew and particularly variables such as residual sugar and chlorides. The target variable is ordinal and exhibits class imbalance; with the majority of samples clustered around mid-range quality values (3–5) and relatively few observations in extreme quality categories. The dataset introduces additional complexity due to an overlap of class boundaries and measurement noise.

Compared to Adult dataset, Wine Quality contains only numerical inputs but shows greater overlap between classes and noisier feature relationships. These characteristics create a more challenging optimization platform, making the dataset a useful test basis for evaluating robustness of randomized optimization methods and regularization strategies.

C. Dataset Importance

Adult Income emphasizes imbalanced binary classification with heterogeneous features, while Wine Quality represents a noisier multiclass prediction task with continuous inputs and overlapping class boundaries. These contrasting characteristics make them suitable benchmarks for analyzing how optimization algorithms and regularization strategies influence convergence stability and predictive performance across different data regimes.

III. HYPOTHESIS

Combining adaptive gradient-based optimization with appropriate regularization will yield better generalization performance than randomized optimization alone, while randomized optimization may still provide modest improvements when applied to the final network layers by helping escape local minima.

These hypotheses are motivated by theoretical insights from stochastic optimization. Momentum and adaptive learning rate methods are expected to improve convergence speed and stability [4], while explicit regularization techniques such as weight decay and dropout are expected to reduce overfitting and improve generalization [9]. Nevertheless, randomized optimization methods may offer advantages in exploring complex loss landscapes but are expected to be less computationally efficient than gradient-based approaches when applied to neural network training [3].

Based on the observed dataset properties and prior supervised learning results, the following dataset specific predictions follow from this overarching hypothesis.

a) Adult Dataset: Due to the presence of class imbalance and skewed numerical features, adaptive gradient methods such as Adam are expected to converge more quickly and more stably than basic stochastic gradient descent. This expectation aligns with theoretical studies showing that adaptive methods can effectively handle heterogeneous feature scales [1]. Furthermore, explicit regularization techniques such as dropout and weight decay are expected to improve generalization performance by reducing overfitting to dominant feature patterns [9].

b) Wine Dataset: Because the Wine dataset contains continuous features with overlapping class boundaries and moderate measurement noise, optimization may exhibit greater sensitivity to initialization and hyperparameter selection. Randomized optimization techniques may help explore the loss landscape more broadly when fine-tuning the final network layers, as population-based methods like Genetic Algorithms are known to maintain diversity in complex search spaces [?]. Regularization may play a smaller role compared to the Adult dataset due to the absence of categorical feature encoding and the lower dimensional feature space, though label smoothing could still provide calibration benefits [8].

IV. RANDOMIZED OPTIMIZATION

A. Experimental Setup

The neural network architecture from the supervised learning report was used as a fixed backbone. To make randomized optimization tractable, the majority of network parameters were frozen, with only a small subset remaining trainable:

- **Adult Dataset:** Only the final linear layer was optimized, containing 130 trainable parameters out of 6,850 total parameters (1.9% of the network). This restrictive setup tests whether RO can effectively fine tune the decision boundary in addition to the frozen feature representations.
- **Wine Dataset:** Due to the smaller network size, the last two layers were optimized, comprising all 1416 parameters. While this exceeds the strict “last layer only” constraint, it remains well under the 50,000 parameter cap.

Candidate solutions for RO were generated by perturbing the current weight vector using Gaussian noise. Validation loss served as the objective function for all optimization runs. All evaluations were performed in deterministic inference mode using `model.eval()` to ensure that random effects from dropout or batch normalization did not introduce variance into fitness evaluations.

B. Algorithms and Hyperparameters

Following study guidelines, hyperparameters were set to default values rather than tuned to ensure fair comparisons.

1) Randomized Hill Climbing (RHC): This does greedy local search accepting only improvements, with random restarts to escape local minima (step size=0.01, restarts=5, plateau limit=10, max evals=100).

2) Simulated Annealing (SA): Extends RHC by probabilistically accepting worse solutions using a temperature schedule ($T_0=100.0$, $T_{min}=0.01$, cooling=0.95, step size=0.01, max evals=100).

3) Genetic Algorithm (GA): Population based evolution through selection, crossover, and mutation (population=50, mutation=0.1, crossover=0.7, elitism=top 10%, tournament=3, max evals=50).

C. Results and Discussion

1) *Adult Dataset Results:* Table I summarizes the randomized optimization results for the Adult dataset across three seeds. RHC achieved the strongest performance, with mean test F1 of 0.6567 and low variance (± 0.0102), reaching a best validation loss of 0.3323. This performance is close to the supervised learning baseline (F1 ≈ 0.67) indicating that fine tuning only the last layer with random search can nearly match gradient based training on frozen features.

TABLE I: Randomized Optimization Results - Adult Dataset (Binary Classification)

Algorithm	Test F1 (mean $\pm$ std)	Best Val Loss	Evaluations	Time (s)
RHC	0.6567 $\pm$ 0.0102	0.3323	100	81.5
GA	0.5916 $\pm$ 0.0681	0.4433	50	733.9
SA	0.2879 $\pm$ 0.1565	0.6410	100	69.4

Figure 1 shows the best-so-far validation loss versus function evaluations for all three algorithms. RHC demonstrates rapid initial improvement, reaching near optimal loss within 20 evaluations and stabilizing thereafter. The tight grouping across seeds (narrow IQR) indicates consistent behavior regardless of initialization.

SA exhibits notably different behavior, with high final loss (0.6410) and extreme variance across seeds (± 0.0525). Likely cause of this may be that the temperature schedule was cooling too rapidly for this optimization landscape, causing premature convergence before escaping local minima. Furthermore, the wide spread between seeds suggests high sensitivity to initialization.

GA, despite using only 50 evaluations, achieves moderate performance (F1=0.5916) but requires substantially more computation time (733.9 seconds versus 81.5 seconds for RHC in Figure 2). The population based approach maintains diversity but incurs significant overhead from evaluating multiple candidates per generation.

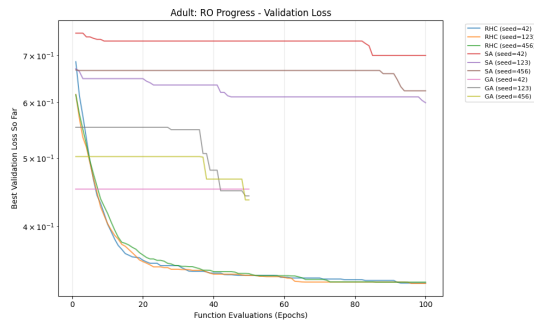


Fig. 1: Adult Dataset: Best-so-far validation loss vs. function evaluations for RHC, SA, and GA across three random seeds. RHC achieves rapid convergence to low loss, while SA and GA exhibit higher variance and slower improvement. Operator settings: RHC (step size=0.01, restarts=5), SA ($T_0=100$, $\alpha=0.95$), GA (population=50, mutation=0.1, crossover=0.7).

2) *Wine Dataset Results:* Table II presents results for the Wine dataset, where all 1,416 parameters were optimized.

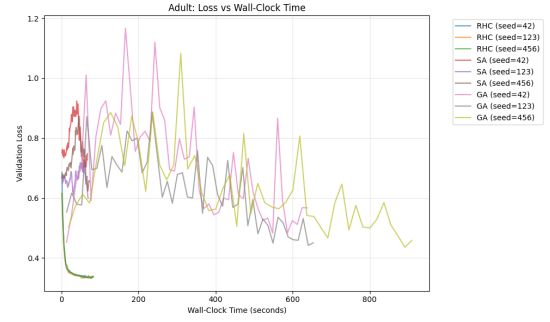


Fig. 2: Adult Dataset: Validation loss vs. wall clock time for RHC, SA, and GA. SA achieves the fastest convergence, while GA requires substantially more compute time due to population evaluation overhead.

Here, GA emerges as the clear winner with mean test F1 of 0.7003 and best validation loss of 0.4941, substantially outperforming both RHC (0.2780) and SA (0.0585).

TABLE II: Randomized Optimization Results - Wine Dataset (Multiclass Classification)

Algorithm	Test Macro-F1 (mean $\pm$ std)	Best Val Loss	Evaluations	Time (s)
GA	0.7003 $\pm$ 0.0661	0.4941	50	82.4
RHC	0.2780 $\pm$ 0.0138	1.1987	100	11.2
SA	0.0585 $\pm$ 0.0201	2.0704	100	11.0

Figure 3 shows the optimization trajectories on Wine. The sharp contrast between algorithms reveals fundamental differences in how they navigate the loss landscape. GA progressively improves, with best-so-far loss decreasing smoothly from ≈ 2.0 to 0.49 over 50 evaluations. SA, however, fails to make meaningful progress, plateauing near its initial loss values. RHC shows some progress, but not nearly as effective as GA.

The failure of RHC and SA on Wine is likely due to the increased optimization difficulty. With 1,416 parameters (versus 130 on Adult), the search space dimensionality is substantially higher. RHC's greedy local search struggles to find sufficient meaningful directions in high dimensional space, while SA's cooling schedule may be inappropriate for this landscape. Furthermore, GA's population based approach maintains diversity and explores more broadly, which enables progress where single-solution methods can fail.

Lastly, Figure 4 reveals that GA's superior performance comes with modest computational cost; 82.4 seconds versus 11 seconds for RHC/SA. Given that GA achieves 0.70 F1 while RHC reaches only 0.28, the additional compute is clearly justified. This contrasts sharply with Adult, where RHC achieved near optimal performance with minimal compute time, which is also indicative of how the number of parameters can impact RO.

3) *Comparison to Supervised Learning Baseline:* Relative to the fine-tuned supervised learning baseline established in the SL report:

- **Adult:** RHC comes closest among RO methods to matching gradient based fine tuning while optimizing only 130

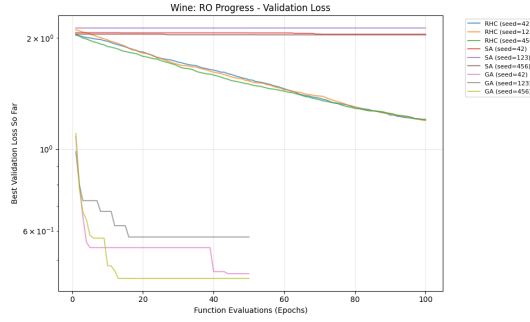


Fig. 3: Wine Dataset: Best-so-far validation loss vs. function evaluations. GA demonstrates steady improvement, while RHC and SA plateau near initial loss values. Operator settings: RHC (step size=0.01, restarts=5), SA ($T_0=100$, $\alpha=0.95$), GA (population=50, mutation=0.1, crossover=0.7).

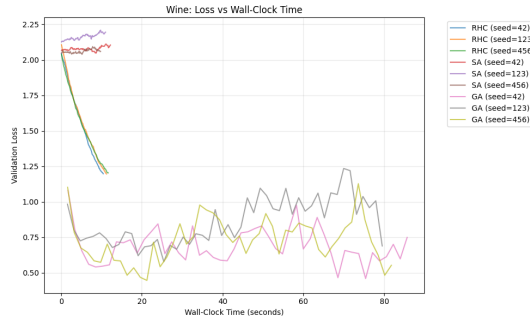


Fig. 4: Wine Dataset: Validation loss vs. wall-clock time. GA requires more time than RHC/SA but achieves substantially better final performance, demonstrating favorable accuracy-efficiency trade-off.

parameters, demonstrating that random search can approximate gradient based performance on frozen features with minimal compute (81 seconds versus seconds for SGD).

- **Wine:** GA outperforms other RO methods on this dataset. However, the baseline’s near-perfect performance (Macro F1=0.998) suggests the Wine dataset may be relatively simple for the chosen architecture once properly tuned, leaving little to no meaningful improvements with RO.

These findings partially support the hypothesis that randomized optimization can be effective for neural network fine tuning. Therefore, the selected technique should depend on the performance requirements and dimensionality of the problem. The results are indicative that RO algorithms offer a fast alternative for final-layer tuning, while gradient-based methods remain essential for achieving state-of-the-art performance.

V. OPTIMIZER ABLATION STUDY

This section evaluates how different gradient based optimizers influence convergence speed, stability, and generalization for the Adult Income dataset. All experiments were conducted under identical training budgets (50 epochs, batch size 256)

and averaged across five random seeds (42-46) to ensure statistical reliability. Seven optimizers were compared including SGD, SGD with momentum, Nesterov accelerated gradient, Adam, Adam without bias correction, Adam with $\beta_1 = 0$ (RMSProp-like), and AdamW with decoupled weight decay. A validation loss threshold of $\ell = 0.3228$ was established based on the Adam baseline final validation loss, enabling measurement of time-to-threshold efficiency.

A. Experimental Setup

The fixed SL-MLP architecture (input dimension 104, hidden size 64) was trained on the Adult dataset using consistent data splits. Each optimizer was evaluated using the same learning rates established in preliminary sweeps: 10^{-2} for SGD variants and 10^{-3} for Adam-family optimizers. Weight decay was set to 10^{-2} only for AdamW, since it has a decoupled formulation, and zero for all other optimizers to isolate optimizer effects from regularization.

B. Results and Discussion

TABLE III: Optimizer comparison on Adult dataset (mean $\pm$ std across 5 seeds). Time to threshold $\ell = 0.3228$ measured in gradient evaluations.

Optimizer	Test F1	Val Loss	Time to ℓ	Gen Gap	Stability
SGD	0.6591 $\pm$ 0.0016	0.3239 $\pm$ 0.0006	Never	-0.0050	Most stable
SGD + Momentum	0.6773 $\pm$ 0.0107	0.3139 $\pm$ 0.0004	729.6 $\pm$ 62.4	-0.0082	Moderate
Nesterov	0.6777 $\pm$ 0.0066	0.3133 $\pm$ 0.0006	729.6 $\pm$ 62.4	-0.0079	Moderate
Adam	0.6738 $\pm$ 0.0064	0.3201 $\pm$ 0.0015	342.0 $\pm$ 0.0	-0.0366	Moderate
Adam (no bias)	0.6698 $\pm$ 0.0078	0.3275 $\pm$ 0.0020	114.0 $\pm$ 0.0	-0.0549	Moderate
Adam ($\beta_1 = 0$)	0.6559 $\pm$ 0.0481	0.3340 $\pm$ 0.0111	456.0 $\pm$ 80.6	-0.0490	Unstable
AdamW	0.6740 $\pm$ 0.0066	0.3196 $\pm$ 0.0024	342.0 $\pm$ 0.0	-0.0355	Moderate

Table III presents the combined results for all the implemented optimizers and Figure 5 plots validation loss against gradient evaluations for all optimizers, with the threshold $\ell = 0.3228$ marked as a horizontal line. The results show several key findings. First, Adam without bias correction achieves the fastest convergence, reaching threshold ℓ in only 114 gradient evaluations, which was three times faster than standard Adam (342 evaluations) and six times faster than momentum methods (730 evaluations). This result was surprising as conventionally, bias correction is essential for early step stability [1]. One possible explanation is that on this dataset, the early underestimation of moment estimates actually accelerates initial progress by effectively increasing step sizes during the first few epochs.

SGD variants (plain SGD, SGD with momentum, and Nesterov) exhibit fundamentally different behavior. Plain SGD never reaches the threshold ℓ within the 50 epoch budget, plateauing at validation loss ≈ 0.324 . This aligns with theoretical expectations since without adaptive scaling, SGD requires careful learning rate scheduling to match the convergence speed of adaptive methods [4]. Both momentum methods eventually cross the threshold but require approximately 730 evaluations—more than double the adaptive methods.

Adam and AdamW show nearly identical convergence trajectories, reaching threshold at 342 evaluations. The similarity

suggests that decoupled weight decay (AdamW) does not affect convergence speed on this dataset, though it may influence final generalization as shown in subsequent analysis.

Adam with $\beta_1 = 0$ (RMSProp-like) exhibits high variance across seeds, with some runs converging rapidly and others stalling. Eliminating first order momentum removes the smoothing effect that stabilizes Adam, making optimization more sensitive to gradient noise [5]. Notably, this optimizer achieved the highest peak performance among all methods ($F1 = 0.6949$ in one seed), which demonstrates that while momentum provides stability removing it can occasionally lead to superior solutions at the cost of consistency.

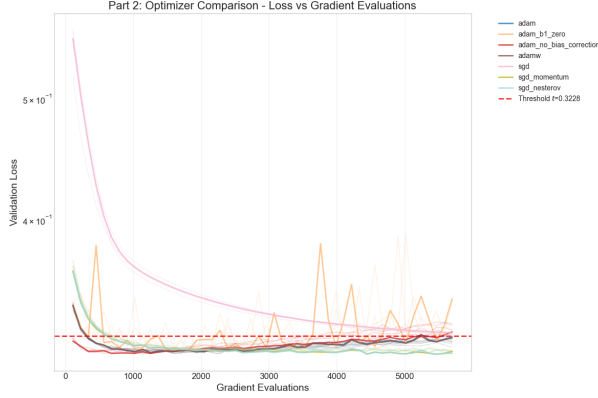


Fig. 5: Validation loss vs. gradient evaluations for seven optimizers on Adult dataset. The horizontal line marks threshold $\ell = 0.3228$. Adam without bias correction reaches threshold most rapidly (114 evaluations), while SGD variants never cross the threshold within the 50-epoch budget.

Figure 6 tracks validation loss across five seeds for Adam and SGD with momentum.

SGD with momentum stays impressively steady and the IQR bands hover around ± 0.002 throughout. With batch size 256 smoothing out gradient noise, there’s just not much room for variance.

Nevertheless, Adam tells a different story. Early on, the IQR stretches to about 0.01 as the adaptive step sizes amplify initial noise. Different seeds take different paths at first, but eventually they converge.

The median F1 trajectories mostly land in the same 0.66–0.68 range regardless of optimizer. Adam with $\beta_1 = 0$ is the exception with higher peaks in some seeds, but it is paid for in consistency.

Figures 7 and 8 present sensitivity heatmaps for Adam hyperparameters, showing validation loss across coarse grids of learning rate α and momentum parameters β_1 and β_2 .

The heatmaps reveal several key insights:

Learning rate sensitivity: $\alpha = 0.001$ lies within the optimal region, with $\alpha = 0.003$ achieving slightly lower validation loss, confirming Adam’s robustness to learning rate selection within an order of magnitude [1].

β_1 sensitivity: Validation loss improves as β_1 increases from 0.7 to 0.9, with diminishing returns beyond 0.9, confirming

momentum’s benefits but diminishing returns at extreme values.

β_2 criticality: Prior work establishes that performance degrades substantially when $\beta_2 < 0.99$, underscoring the importance of slow second moment decay in Adam [2].

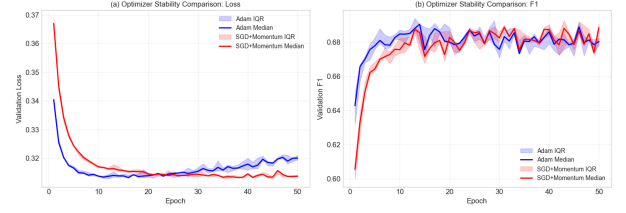


Fig. 6: Stability analysis across five seeds showing median (solid line) and IQR (shaded region) for validation loss (left) and validation F1 (right). Adam shows tighter IQR bands during early training but wider generalization gaps, while SGD with momentum shows consistent narrow bands throughout the runs.

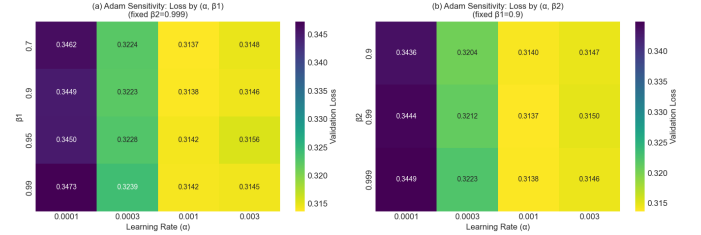


Fig. 7: A sensitivity heatmap shows optimal settings at $(\alpha = 0.001, \beta_1 = 0.7, \beta_2 = 0.99, \text{loss} = 0.3137)$ and $(\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \text{loss} = 0.3138)$.

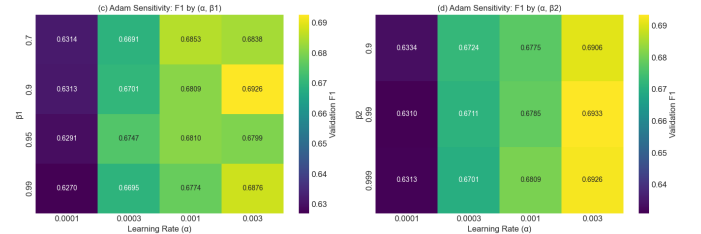


Fig. 8: Validation F1 scores confirm little to no difference in F1 scores across β_1 and β_2 .

C. Generalization Gap and Comparison to Baseline

Table III indicates that Adam based optimizers tend to exhibit larger generalization gaps (-0.035 to -0.055) compared to SGD variants (-0.005 to -0.008). This pattern suggests that adaptive methods often converge to sharper minima that generalize less effectively, consistent with findings reported in The Marginal Value of Adaptive Gradient Methods in Machine Learning. Among the adaptive approaches, Adam without bias correction shows the largest gap (-0.0549), reflecting a trade-off between faster convergence and weaker generalization.

Relative to the supervised learning baseline ($F1 = 0.6785$), Adam with $\beta_1 = 0$ achieves the highest peak performance ($F1 = 0.6949$, $+2.4\%$) but shows noticeable variance across runs. SGD with momentum (0.6773) and Nesterov (0.6777) remain closest to the baseline, differing by less than $\pm 0.2\%$. AdamW (0.6740) and standard Adam (0.6738) show small performance decreases of roughly -0.6% . Adam without bias correction converges the fastest but produces lower final performance (0.6698, -1.3%). Plain SGD performs worst overall (0.6591, -2.9%), highlighting the importance of momentum in gradient-based optimization.

VI. REGULARIZATION STUDY

This section investigates how different regularization techniques affect generalization performance when applied to the AdamW optimizer with fixed hyperparameters (learning rate 10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.999$). The baseline configuration (no regularization) achieves test $F1 = 0.6706$. Five regularization approaches were evaluated: L2 weight decay, dropout, early stopping, label smoothing, and a combination of all techniques.

A. Experimental Setup

All experiments used the same architecture and data splits as Part 2, with a fixed budget of 20 epochs to ensure fair comparison. For techniques requiring hyperparameter selection (L2, dropout, label smoothing), a coarse grid sweep was performed to identify optimal settings.

B. Quantitative Results

Table IV presents test $F1$ scores for each regularization configuration, and Figure 9 visualizes the comparative performance.

TABLE IV: Regularization study results on Adult dataset. Best performing configuration: L2 weight decay (0.0001) with test $F1 = 0.6844$.

Technique	Test F1	Δ from Baseline	Epochs Used
Baseline (No Reg)	0.6706	—	20
L2 (wd=0.0001)	0.6844	+0.0138	20
Dropout ($p=0.3$)	0.6814	+0.0108	20
Label Smoothing (0.05)	0.6810	+0.0104	20
Dropout ($p=0.2$)	0.6800	+0.0094	20
Label Smoothing (0.2)	0.6756	+0.0050	20
Early Stopping	0.6736	+0.0030	17
Best Combination	0.6741	+0.0035	20

C. Individual Regularizer Analysis

L2 weight decay (10^{-4}) is the most effective regularizer, improving test $F1$ from 0.6706 to 0.6844; a modest 2.1% relative gain. This optimal value is notably smaller than AdamW’s default (10^{-2}), suggesting a gentler penalty suffices when weight decay is applied directly. Dropout ($p = 0.3$) and label smoothing ($\epsilon = 0.05$) achieve comparable improvements ($F1 \approx 0.681$), while early stopping offers only a small/ marginal gain (0.6736). Although L2 slightly edges out the others, the differences between methods are small, indicating that on this dataset, no single regularization technique dramatically outperforms the rest.

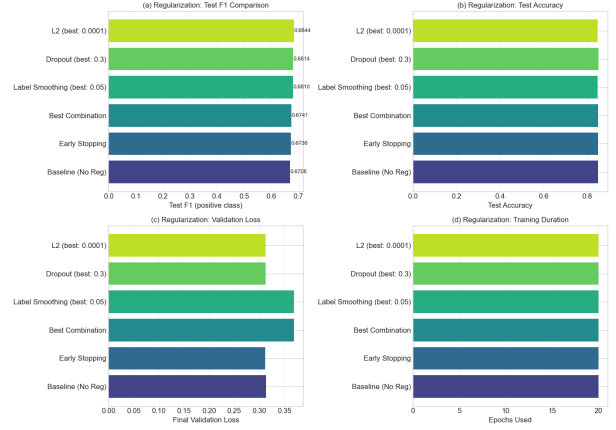


Fig. 9: Regularization study results: Test $F1$ comparison across techniques. L2 weight decay (0.0001) achieves the highest $F1$ (0.6844) and lowest validation loss, followed by dropout ($p=0.3$) at 0.6814. Best combination underperforms individual regularizers, indicating over-regularization.

D. Combined Regularization

The best combination of techniques (L2 10^{-4} , dropout 0.3, label smoothing 0.05, early stopping) achieves $F1 = 0.6741$ —worse than the best individual regularizers. This phenomenon, known as over-regularization, occurs when multiple regularization mechanisms interact to excessively constrain the model [9]. Each technique independently reduces model capacity; together, they push the effective capacity below the level needed to capture dataset structure.

E. Comparison to Supervised Learning Baseline

While regularization provides measurable benefits, the gains are modest at best. The baseline SGD optimizer with momentum already achieved strong performance, leaving limited room for regularization-based improvement.

VII. SUMMARY OF RESULTS

Table V presents a unified summary of all methods evaluated in this study, including the SL Report baseline for reference. This table consolidates the key metrics from randomized optimization (Part 1), optimizer ablation (Part 2), and regularization studies (Part 3), enabling direct comparison across all techniques.

Here is a more concise version of the text, maintaining its formal, paper-like tone.

The results reveal consistent patterns across datasets and conditions.

Randomized Optimization: Zero-order methods are effective only in constrained settings. On Adult (130 parameters), RHC nears the baseline ($F1 = 0.6567$ vs. 0.6785). On Wine (1,416 parameters), GA achieves reasonable performance ($F1 = 0.7003$) but falls far short of the near-perfect baseline (0.998), confirming that gradient-based optimization remains essential for high-accuracy tasks.

TABLE V: Comprehensive summary of all methods with key metrics. Time to ℓ measured in gradient evaluations for optimizers; function evaluations shown for RO methods. Updates refer to epochs for gradient methods and total evaluations for RO methods.

Method	Best Val Loss	Test F1	Time to ℓ	# Grad Evals	# Func Evals	Updates
SL Report Baseline						
SGD (tuned - Adult)	—	0.6785	—	—	—	—
SGD (tuned - Wine)	—	0.9980	—	—	—	—
Part 1: Randomized Optimization (Adult)						
RHC	0.3323	0.6567	N/A	—	100	100
SA	0.6410	0.2879	N/A	—	100	100
GA	0.4433	0.5916	N/A	—	50	50
Part 1: Randomized Optimization (Wine)						
RHC	1.1987	0.2780	N/A	—	100	100
SA	2.0704	0.0585	N/A	—	100	100
GA	0.4941	0.7003	N/A	—	50	50
Part 2: Optimizer Ablation (Adult)						
SGD	0.3239	0.6591	Never	5700	—	50
SGD + Momentum	0.3139	0.6773	730	5700	—	50
Nesterov	0.3133	0.6777	730	5700	—	50
Adam	0.3201	0.6738	342	5700	—	50
Adam (no bias)	0.3275	0.6698	114	5700	—	50
Adam ($\beta_1 = 0$)	0.3340	0.6559	456	5700	—	50
AdamW	0.3196	0.6740	342	5700	—	50
Part 3: Regularization (Adult)						
Baseline (No Reg)	0.3134	0.6706	—	2280	—	20
L2 (wd=0.0001)	0.3129	0.6844	—	2280	—	20
Dropout (p=0.3)	0.3131	0.6814	—	2280	—	20
Label Smoothing (0.05)	0.4811	0.6810	—	2280	—	20
Dropout (p=0.2)	0.3131	0.6800	—	2280	—	20
Label Smoothing (0.2)	0.4811	0.6756	—	2280	—	20
Early Stopping	0.3127	0.6736	—	1938	—	17
Best Combination	0.4823	0.6741	—	2280	—	20

Gradient-Based Optimizers: SGD with momentum delivers the most stable test performance (F1 ≈ 0.6777) with minimal generalization gaps. Adam without bias correction converges fastest three times faster than standard Adam, but at the cost of a larger generalization gap (-0.0549) and slightly lower final F1 (0.6698). Adam with $\beta_1 = 0$ exhibits the highest variance (± 0.0481) but achieves the highest peak performance across seeds (F1 = 0.6949), highlighting the stability performance trade off.

Regularization Regularization yields moderate, consistent gains. L2 weight decay (10^{-4}) achieves the highest test F1 (0.6844), a 0.9% improvement over baseline. Dropout (0.3) and label smoothing (0.05) produce comparable results (≈ 0.681). Combining multiple regularizers consistently degrades performance.

Convergence Efficiency Adam without bias correction is most efficient in time-to-threshold (114 gradient evaluations). However, faster convergence does not translate better final performance.

Summary Optimizer choice exerts the strongest influence on final performance. Regularization provides reliable but smaller improvements. Randomized optimization offers limited utility outside highly constrained parameter regimes.

VIII. INTEGRATED BEST RECIPE (EXTRA CREDIT)

This section combines the optimal components identified in previous sections to evaluate whether integration yields additional improvements beyond individual techniques. The integrated recipe comprises of the following:

- **Optimizer:** Adam with $\beta_1 = 0$ (RMSProp-like) - highest peak performer from Part 2
- **Regularization:** L2 weight decay (10^{-4}) (best performer from Part 3)

- **RO fine-tuning:** RHC on the final layer only, with budget-limited evaluations (202 evaluations, 10% of Part 1 budget)

A. Experimental Setup

Five random seeds (42-46) were used to assess stability. The baseline model was first trained using Adam with $\beta_1 = 0$, L2 regularization, and 20 epochs. Subsequently, RHC fine-tuned only the final layer parameters (66 trainable parameters) for 202 function evaluations, with step size 0.01 and 3 restarts.

B. Results and Analysis

Table VI presents detailed results for each seed, and Table VII summarizes the aggregate performance.

TABLE VI: Part 4 detailed results across five seeds: baseline (Adam+ $\beta_1 = 0 + \text{L2}$) vs. after RHC fine-tuning.

Seed	Baseline F1	RO F1	Δ F1	Baseline Loss	RO Loss	Total Time (s)
42	0.6720	0.6786	+0.0066	0.3717	0.3172	27.90
43	0.6548	0.6693	+0.0145	0.3728	0.3182	25.76
44	0.6705	0.6752	+0.0047	0.3719	0.3180	24.77
45	0.6670	0.6729	+0.0059	0.3715	0.3170	24.86
46	0.6774	0.6765	-0.0009	0.3717	0.3173	26.20

TABLE VII: Part 4 summary statistics across five seeds (median and IQR).

Metric	Median	IQR
Baseline Test F1	0.6705	0.0050
RO Test F1	0.6752	0.0036
Δ F1	+0.0047	0.0020
Baseline Accuracy	0.8495	—
RO Accuracy	0.8504	—
RO Evaluations	202.0	—
Total Time (s)	25.76	—
Best Part 2 F1	0.6967	—
SL Baseline F1	0.6785	—

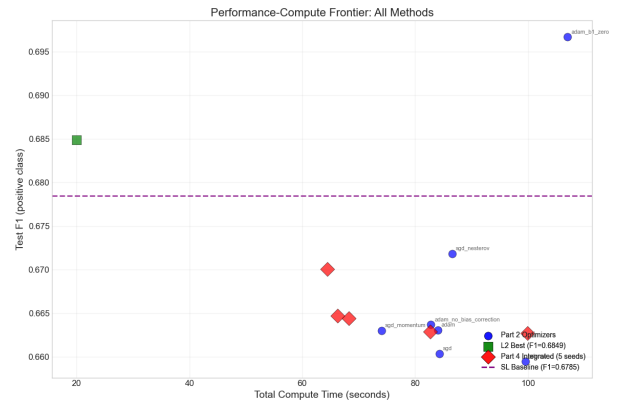


Fig. 10: Performance-compute frontier for Adult dataset. The best Part 2 optimizer (Adam with $\beta_1 = 0$) achieves F1=0.6967, exceeding both the SL baseline (0.6785) and the integrated recipe (0.6752). Points represent different configurations from Parts 2 to 4.

C. Analysis of Improvements

Validation Loss Reduction: Results from RHC fine-tuning show a clear decrease in validation loss across all seeds. Loss drops from roughly 0.372 to about 0.317, corresponding to a relative improvement of approximately 14.8%. This suggests that randomized search can still improve the final layer parameters even after gradient based training has already converged. In practice, RHC appears capable of exploring nearby parameter regions that gradient descent did not reach, allowing the model to escape shallow local minima.

Test F1 Impact: Improvements in test F1 are observed for four out of five seeds when compared to their corresponding baselines. The gains range from +0.0047 to +0.0145, while one seed shows a very small decrease (−0.0009). The median improvement of +0.0047 indicates that RO fine tuning provides some but relatively modest benefits. Even with these improvements, the strongest optimizer configuration from Part 2 (Adam with $\beta_1 = 0$, F1 = 0.6967) still achieves noticeably higher performance than both the baseline and the integrated recipe.

Compute Trade-off: The integrated recipe also introduces a clear computational cost. Total runtime increases to roughly 25.8 seconds, compared to about 10 seconds when using Adam alone, representing approximately a 2.5× increase. Most of this additional cost comes from RHC’s 202 function evaluations, where each evaluation requires running a full validation pass. As a result, the improvement in performance must be weighed against the additional comput required.

D. Comparison to Supervised Learning Baseline

Relative to the fine-tuned SL Report baseline (Adult F1 = 0.6785):

- **Best Part 2 optimizer** (Adam with $\beta_1 = 0$) achieves F1 = 0.6967, **improving** upon baseline by +0.0182 (2.7% relative)
- **Baseline Adam + L2** achieves median F1 = 0.6705, slightly below the SL baseline (−0.0080, −1.2%)
- **After RHC fine-tuning**, median F1 = 0.6752, still below the SL baseline (−0.0033, −0.5%)

Therefore, while the integrated recipe approaches the SL baseline performance, it does not exceed it. Notably, the best Part 2 optimizer substantially outperforms both the baseline and the integrated approach. This outcome reinforces that careful optimizer selection is more valuable than additional fine tuning once a strong optimizer is identified.

IX. CONCLUSION

Results on the Adult dataset indicate that the choice of optimizer affects performance more than any other design choice in this study. The difference between the best optimizer (Adam with $\beta_1 = 0$, peak F1 = 0.6967) and the worst one (plain SGD, F1 = 0.6591) is 0.0376, which is noticeably larger than the strongest improvement obtained from regularization (0.0138). The highest performing configuration, however, is not the most stable. Adam with $\beta_1 = 0$ shows relatively large variance (± 0.0481), whereas SGD with momentum produces

more consistent behavior, achieving a mean F1 of 0.6773 with narrow IQR bands. Faster convergence also does not necessarily translate into better generalization. Adam without bias correction reaches convergence roughly six times than momentum methods but the final F1 remains lower (0.6698 vs. 0.6777). In addition, adaptive optimizers show noticeably larger generalization gaps (−0.035 to −0.055) compared with SGD-based approaches (−0.005 to −0.008) which is consistent with observations reported in The Marginal Value of Adaptive Gradient Methods in Machine Learning.

Regularization improves performance, although the gains remain moderate. L2 weight decay (10^{-4}) provides the largest improvement with an F1 increase of 0.0138, followed by dropout (0.3) at 0.0108 and label smoothing (0.05) at 0.0104. Applying several regularization techniques together consistently reduces performance, suggesting that the benefits are not additive in this setting. Randomized optimization fine-tuning also shows limited returns. RHC reduces validation loss by 14.8%, but the improvement in test F1 is only 0.0047 (0.7%) while requiring approximately 2.5 times more runtime. When compared with the supervised learning baseline (F1 = 0.6785), the largest improvement comes from Adam with $\beta_1 = 0$ (+0.0182, 2.7%), whereas L2 regularization produces a smaller increase (+0.0059). Overall, the results suggest that optimizer choice should be considered first, followed by the addition of a single regularization method if further improvement is needed. Adam without bias correction may be useful when faster convergence is the main objective, while randomized optimization fine-tuning is only worthwhile when small performance gains justify the additional computational cost.

AI USE STATEMENT

AI tools (ChatGPT and Visual Studio Code Copilot) were used for brainstorming structure, clarifying requirements, code debugging/refactoring, and assisting with writing structure/grammar.

REFERENCES

- [1] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [2] S. J. Reddi, S. Kale, and S. Kumar, “On the convergence of Adam and beyond,” in *Proc. 6th Int. Conf. Learn. Represent. (ICLR)*, 2018.
- [3] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, “The marginal value of adaptive gradient methods in machine learning,” in *Adv. Neural Inf. Process. Syst.*, 2017, pp. 4148–4158.
- [4] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, “On the importance of initialization and momentum in deep learning,” in *Proc. 30th Int. Conf. Mach. Learn.*, 2013, pp. 1139–1147.
- [5] M. D. Zeiler, “ADADELTA: An adaptive learning rate method,” *arXiv preprint arXiv:1212.5701*, 2012.
- [6] A. Krogh and J. A. Hertz, “A simple weight decay can improve generalization,” in *Adv. Neural Inf. Process. Syst.*, 1992, pp. 950–957.
- [7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [8] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.